

Snip: An Integrated DevOps Pipeline

Project Report — [Riccardo Fragale, Lorenzo Deflorian {fragale,ldef}@kth.se]

8 October 2026

1 Overview

Snip is a URL shortener with a FastAPI backend, a dependency-free HTML/CSS/JavaScript frontend, and a SQLAlchemy persistence layer. Users create short links, follow redirects, inspect redirect counts, and disable links while retaining statistics. Although the application is small, its durable data and schema migrations make it suitable for demonstrating a complete DevOps workflow, from reviewed source changes to a verified deployment.

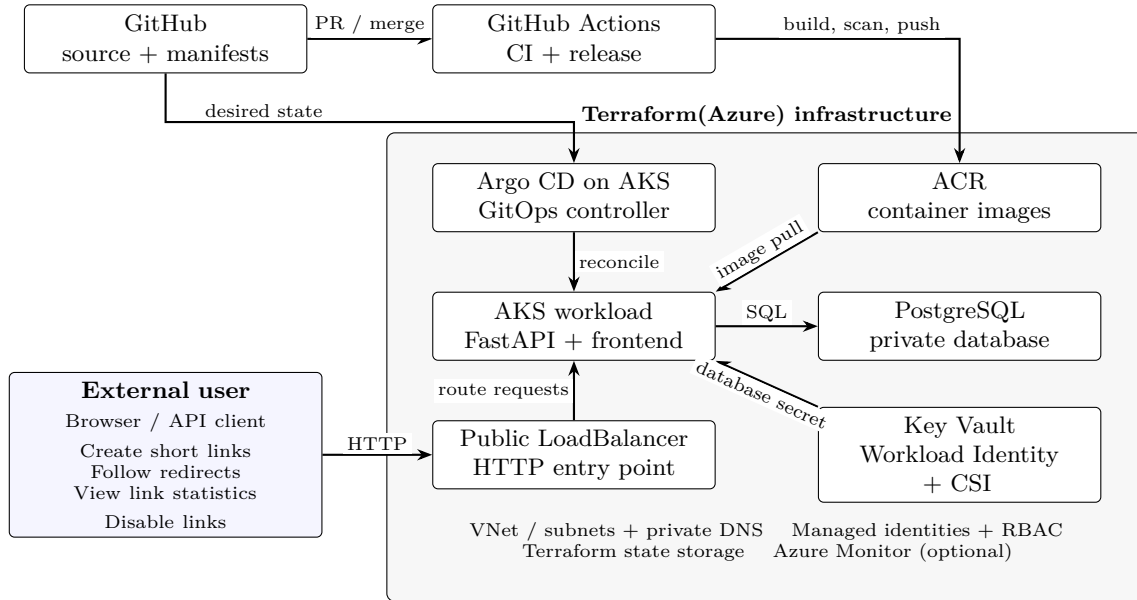


Figure 1: Delivery and runtime architecture. Terraform provisions the Azure infrastructure grouped inside the boundary; Argo CD and the application run on AKS, and the Kubernetes Service creates the public LoadBalancer. External users access the application through that LoadBalancer. The release workflow also commits the selected image SHA back to GitHub.

The whole application was deployed onto Azure platform thanks to the free credits given to student accounts through KTH email. The backend serves the API and frontend from one container, simplifying deployment. Azure Kubernetes Service (AKS) runs the pods, Azure Container Registry (ACR) stores images, and managed PostgreSQL 16 holds data outside the cluster. A public LoadBalancer routes browser requests to the application. Public access to the application API and frontend was chosen so everyone can access it and grade our work. PostgreSQL has public access disabled and is reached through a delegated subnet and private DNS. Key Vault supplies the database connection through Azure Workload Identity and the Secrets Store CSI driver.

2 CI pipeline

Pull requests trigger a reusable GitHub Actions workflows for application quality, security, and configuration validation. The workflow installs Python 3.12 and uses `uv sync --locked` to reproduce dependencies from `uv.lock`. It runs Ruff formatting and lint checks, `ty` type checking, and tests through `pytest`. The security workflow also builds the Docker image, making container build failures visible before a merge in order to understand earlier what could happen in production.

Unit tests check short-code generation. Integration tests cover API behavior, migrations, persistence across sessions, code-collision recovery, and redirect-counter updates from stale sessions. These tests use SQLite, which keeps CI as quick as possible but leaves PostgreSQL-specific behavior untested. The exact source commit on `main` passes the same quality, security, and configuration workflows again before release. This connects CI results to the source used to build the delivered image.

3 CD pipeline

A push to `main` starts the CD workflow contained in `.github/workflows/release.yml`. After all validation jobs succeed (the ones mentioned before in the CI), the release job authenticates to Azure with OIDC, builds a source-SHA-tagged image, scans it, and pushes that same image to ACR.

A dedicated GitHub App obtains a temporary installation token and commits the image tag and `GIT_SHA` to the production Kustomize overlay. This Git commit records the selected application version for Argo CD.

Release jobs are serialized.

A current-head check rejects already-stale source commits, and a normal push fails if `main` advances rather than overwriting newer changes. Pushes changing only the production overlay skip the release workflow, preventing release loops and allowing an existing image to be selected for rollback.

SHA tags improve traceability, although digest pinning and registry-enforced immutability are not configured. The mutable base image and OS package updates also mean rebuilding a source SHA may produce different image bytes.

Argo CD watches `main` at `k8s/overlays/production`. Automatic synchronization, pruning, and self-healing reconcile Kubernetes with Git. Namespace, ServiceAccount, and SecretProviderClass are applied at sync waves `-3` and `-2`. The migration Job is a `Sync` hook at wave `-1`: it runs `alembic upgrade head` before the default-wave Deployment. A failed migration prevents the later rollout wave from proceeding.

The production overlay specifies two replicas and a rolling update with one surge pod and zero unavailable pods. Liveness checks process availability; readiness queries the database and removes unready pods from Service traffic. Once the deployment is healthy, a `PostSync` Job creates a link, checks its HTTP 307 redirect destination, and disables it. Hook failures are visible in Argo CD but do not automatically revert the application. Actions finishes after publishing and committing desired state, so delivery success and deployment success are observed separately. Reverting the overlay can select an earlier image, but does not undo database migrations. Schema changes therefore need compatibility with old pods during rollout and rollback. Destructive migrations require a separate recovery plan. Smoke-test cleanup disables its temporary link rather than deleting the database row, so releases accumulate inactive records.

4 Infrastructure as Code

`infrastructure/terraform` provisions AKS, ACR, virtual networking, private PostgreSQL, Key Vault, identities, and optional Azure Monitor integration. AKS uses RBAC. Terraform supports API-server IP restrictions but also permits an empty allowlist, which allows access from any IP to the AKS management API while still requiring authentication and RBAC authorization. This management endpoint is separate from the public application API used to try the URL shortener. Its kubelet identity receives `AcrPull`, while a separate federated workload identity receives `Key Vault Secrets User`. CSI mounts the connection secret and mirrors it into the Kubernetes Secret consumed as `DATABASE_URL`. Rotation refreshes CSI content, but running containers need a restart to refresh environment variables.

Terraform makes cloud changes reviewable and repeatable; Kustomize keeps shared workload definitions in `k8s/base` and production overrides in a separate overlay. AKS and Argo CD add operational complexity but demonstrate reconciliation and rollout control directly.

Managed PostgreSQL reduces storage and backup administration at extra cost. The default one-node AKS pool limits availability: two replicas help rolling updates but cannot survive node loss.

5 Development platform

GitHub hosts application code, infrastructure definitions, deployment manifests, and documentation in one repository. The intended process uses feature branches, reviewed pull requests, and required CI checks before merging to `main`. Branch protection and the release App's narrowly scoped bypass require repository settings; their live configuration cannot be verified from tracked files alone. The App needs a stored private key, while Azure authentication uses OIDC without an Azure client secret.

The layout separates HTTP routes, business logic, and persistence in `app/api`, `app/services`, and `app/db`. Migrations and tests accompany the application; workflows and manifests describe its delivery. Keeping source and desired state together simplifies the course submission, at the cost of release-bot commits on `main`. Local development uses `./run.sh` to install locked dependencies, migrate SQLite, and serve the application without Azure credentials.

6 Security and Quality gates

Quality and security automation checks code, dependencies, images, and infrastructure before release. Ruff enforces formatting and lint rules, `ty` checks types, and `pytest` verifies behavior. `config-validation.yml` checks Terraform formatting, initializes providers without a cloud backend, runs validation and TFLint, and validates rendered manifests with `kubeconform`. `security.yml` runs Gitleaks on repository history and Trivy on dependencies, Terraform configuration, and container images. Configured HIGH/CRITICAL findings fail the jobs; the release image is scanned again before publication. Dependabot proposes weekly Python and Actions updates. Runtime controls include a non-root container, resource requests/limits, private PostgreSQL, restricted Key Vault access, and AKS RBAC. AKS management API access can be restricted by IP, but an empty allowlist removes that network restriction. Structured logs, request IDs, and `/version` support traceability.

7 Key design decisions

The API and browser frontend share one stateless container to simplify deployment and support multiple replicas, while managed PostgreSQL keeps durable data independent of the application pods and reduces backup administration. AKS was chosen to demonstrate Kubernetes rollout control and GitOps, despite its greater complexity than simpler hosting options. Terraform defines Azure infrastructure, while Kustomize defines workloads and production settings, separating cloud provisioning from application releases. Security combines CI scans with runtime controls: OIDC avoids a stored Azure client secret, Workload Identity grants scoped Key Vault access, and private database networking reduces public exposure. Argo CD makes Git the source of desired deployment state: reviewed manifests and SHA-tagged image references provide an auditable version history, automatic synchronization delivers changes, and self-healing repairs drift. Actions publishes artifacts and updates Git without deploying directly to the cluster. Ordered migration and smoke-test hooks check the database before rollout and application behavior afterward.

8 Limitations

The single-node AKS configuration reduces cost but cannot provide availability during node failure, even with two replicas. Finite Azure credits also limit continuous availability. Public access uses HTTP without TLS, authentication, or rate limiting; clients can create or disable links, and the application uses database administrator privileges. Application NetworkPolicy resources and explicit pod-level hardening are absent. SQLite-based CI tests do not verify PostgreSQL-specific behavior, image scans ignore unfixed vulnerabilities, and custom resources with missing schemas escape full validation. SHA tags provide traceability without enforcing image immutability. Argo CD reports deployment failures without automatic rollback; restoring an earlier image does not reverse database migrations. Secret rotation requires container restarts. Service-level objectives, alert rules, and validated disaster-recovery procedures are absent. TLS, restricted database roles, rate limiting, and network isolation are priorities before broader use.

9 AI tools usage

We have used AI agents for code reviews, debugging and to help us understand the terraform and azure documentation. The entire url-shortener app (folder app) has been entirely AI generated as it is not the focus of the current project. LLMs were also used to write part of the documentation and the README file.

10 How to test and evaluate

To examine CI in more detail, read the triggered GitHub Actions workflows in `.github/workflows: typecheck.yml`, `security.yml`, and `config-validation.yml` define the pull-request checks, while `release.yml` calls these checks again on pushes to `main` before publishing a release. Their event triggers, jobs, and commands specify the pipeline; then the runs we've made during the development could be seen on the github repo online with executions and logs.

The implementation and configuration(especially for the infrastructure side) can be inspected directly in the repository: Terraform state bootstrap is in `infrastructure/bootstrap-state`, and Azure resource definitions are in `infrastructure/terraform`. Kubernetes resources are in `k8s/base`, production overrides are in `k8s/overlays/production`, and the Argo CD application is specified in `argocd/application.yaml`.

In order to verify that the Application is up and to curl the url shortener we kindly remind the grader to send us a mail so that we will ensure that the application is up (as we are spending credits to deploy an Azure) and we will tell you the url where to find the deployed application.